

Midterm Project: Predicting Student Performance

Due: 11:59 p.m., October 5, 2026

Overview

Assume you are working as a data scientist for the AlamoGreat Independent School District (AISD). AISD is adopting a data-driven approach to improving student performance and has two goals. First, it wants a model that predicts each student's performance, so that staff can reach out early to students who may need help. Second, it wants to know which factors are most closely associated with a student's overall performance.

In this project, you will predict each student's Performance Index from study habits and prior scores, then identify the three most important features. The project has three parts, covering data preparation, a from-scratch implementation of linear regression with gradient descent, and a computational analysis of feature importance.

By the end of this project, you will be able to do the following.

- Derive and implement gradient descent for a multi-feature linear model.
- Clean messy data and build a reproducible, leak-free pipeline.
- Predict students' Performance Index on a held-out test set.
- Measure feature importance computationally and support a claim with more than one kind of evidence.

Part 1: The Data

You are given two files. `train.csv` contains the features and the target. `test_features.csv` contains the features and an `ID` column, but no target. Develop and select your model using `train.csv` only. Then apply your final model to `test_features.csv` and submit the predicted Performance Index for every student (see Part 2 for the file format).

Column	Description
Hours Studied	Hours spent studying
Weekly Study Hours	Self-reported total study hours per week
Previous Scores	Score on previous exams (0 to 100)
Extracurricular Activities	1 = participates, 0 = does not
Sleep Hours	Average hours of sleep per night
Sample Question Papers Practiced	Number of practice papers completed
Commute Minutes	Typical one-way commute to school
Performance Index	Target. Overall performance score (0 to 100). <i>Not included in the test file.</i>

Data cleaning. Like most real-world data, `train.csv` contains missing values and data-entry errors. Missing values must be handled before training because they will break gradient descent. Two common approaches are (1) removing incomplete rows, which shrinks the training set, and (2) imputing missing values, for example with the column mean or median. Data-entry errors are values that are impossible or implausible for the column; inspect the data to find them and decide how to handle them.

Normalization or standardization. The features are on very different scales. Used as-is, large-valued features dominate the gradient, which makes gradient descent slow or unstable. Normalization (rescaling to a fixed range) and standardization (rescaling to zero mean and unit variance) put the features on a common scale. Compute any scaling statistics from the training data only, and apply the same statistics to the validation and test data. Though normalization or standardization is not required, it is recommended to do one to improve model learning.

Document every data-handling decision in Section 2 of your report (see Report Requirements).

Part 2: Linear Regression and Gradient Descent from Scratch

Extend the class demos from one or two features to all features in this dataset. Use NumPy only, and minimize the same cost function used in the class demos.

Tasks

- **Implement a class** `LinearRegressionGD` with `fit(X, y)` and `predict(X)`. The constructor takes a learning rate, a maximum number of iterations, and a convergence tolerance. The class records the cost at every iteration.
- **Create a validation set.** Because the test labels are withheld, split `train.csv` into training and validation sets and use the validation set for all model selection.
- **Fully train the model on the *training set*, and select the best model based on the *validation set*.** You may tune the learning rate, engineer new features, remove features, and so on.
- **Predict the test set** with your final `LinearRegressionGD` model. Predictions from any other model (for example, scikit-learn) receive no performance credit.
- **Submit** `predictions.csv` with your final model's prediction for every student in `test_features.csv`.
 - The file needs to contain exactly two columns, with the header `ID,Performance Index`
 - One row per row of `test_features.csv`, with the same IDs in the same order
 - Numeric predictions (decimals are fine), no missing values, no index column

The first few lines should look like this (*note: the performance index values are for illustration purposes, not the real value you will get*).

```
ID,Performance Index
1,43.2
2,27.8
3,68.5
```

There are many ways to create the `.csv` file. Below is one example of how to write the file with pandas.

```
submission = pd.DataFrame({"ID": test["ID"], "Performance Index": predictions})
submission.to_csv("predictions.csv", index=False)
```

Part 3: The Three Most Important Features

Identify the three most important features for predicting Performance Index, in ranked order. Your ranking must come from computed results, not from intuition or assumptions about what should matter. You may use scikit-learn in this part (for example, for tree-based importance); Part 2 remains NumPy only.

The table below lists methods you can use. *This is not an exhaustive list. Feel free to use methods that are not mentioned in the list.*

Method	How it works	What it tells you
Permutation importance	Shuffle one feature's values in the validation set and measure how much validation RMSE increases. Repeat for each feature.	How much the trained model relies on that feature
Drop-one-feature retraining	Retrain the model without one feature and compare validation RMSE with the full model. Repeat for each feature.	How much predictive information is lost without that feature
Single-feature models	Train a model on one feature at a time and compare validation RMSE.	How much each feature can predict on its own
Standardized coefficients	Fit a linear model on standardized features and compare the absolute coefficient sizes.	Each feature's linear effect per standard deviation
Tree-based importance	Read <code>feature_importances_</code> from a random forest or gradient-boosted model.	How much each feature reduces error in the trees' splits

Requirements

1. **Use at least two different methods** from the table, or others you can justify, such as SHAP values.
2. **Report the numbers.** Include a table with each feature's score under each method, and base your ranking on those scores.
3. **Compare the methods.** State whether they produce the same ranking. Where they disagree, explain why.
4. **Check stability.** Repeat the analysis with at least three different random train/validation splits and report whether the ranking holds.
5. **Examine the relationships.** Plot each top feature against the target. Check whether any features are strongly correlated with each other, and explain how that correlation affects each method.
6. **Distinguish prediction from causation.** In one paragraph, state whether your results show that changing a feature would change a student's score, and what this data can and cannot support.

Report Requirements

The report may be up to 8 pages, excluding references. Use the section headings below, in this order, so that each graded item is easy to locate. Label every figure and table and refer to it in the text. Each bullet is a graded item; missing items receive no credit.

1. Introduction (about half a page)

- The prediction task and AISD's two goals
- Your final validation MSE and your top three features

2. Data Handling

- 2.1 Exploration. A table of summary statistics (minimum, maximum, mean) and the number of missing values in each column.
- 2.2 Missing values. The method used, the number of affected rows or cells, and why you chose it.
- 2.3 Data-entry errors. How you detected them (the rule or plot used), how many you found in each column, how you handled them, and why.
- 2.4 Scaling. The method used, why you chose it, and a statement that its statistics came only from the training data. If you decided NOT to scale the data, explain why.
- 2.5 Pipeline. The order of preprocessing steps and how the same steps were applied to `test_features.csv`.

3. Model Development

- 3.1 Implementation. The cost function, the gradient update rule for all parameters, and the stopping rule, written as equations.
- 3.2 Validation design. The split ratio and random seed.
- 3.3 Training behavior. Cost-versus-iteration curves for at least two learning rates, the learning rate you chose, and the number of iterations to convergence.
- 3.4 Model selection. A table of experiments listing the feature set, the learning rate, and the validation MSE for each. Explain any engineered features and why you tried them.
- 3.5 Final model. The final settings, whether you retrained on the full training file, and the final validation RMSE.

4. Feature Importance

- 4.1 Methods. A brief description of each method used (at least two).
- 4.2 Results. A table showing each feature's score under each method, plus your ranked top three.
- 4.3 Comparison. Where the methods agree and disagree, and why.
- 4.4 Stability. Rankings from at least three random splits, and whether they hold.
- 4.5 Relationships. Plots of each top feature against the target, a correlation matrix (table or heatmap) of the features, and how correlated features affect each method.
- 4.6 Prediction vs. causation. One paragraph, as described in Part 3.

5. Conclusion (a few sentences)

- What AISD should take from your results, and one limitation of your analysis

References

- Any tutorial, paper, or code you adapted, and a brief statement of how you used AI assistants, if at all

Deliverables

Submit three files by the deadline.

File	Contents
report.pdf	A report that follows the Report Requirements
linear_gd.py	Your <code>LinearRegressionGD</code> class and the code that produces <code>predictions.csv</code>
predictions.csv	Columns <code>ID</code> and <code>Performance Index</code> ; one row per test ID

Rules

- AI assistants.** You may use AI assistants to explain concepts and debug errors. You may be asked to explain any line of your code in a short check-in.
- Sources.** Cite any tutorial, paper, or code you adapted.
- Test set.** Use the test set only to produce your final predictions. Any tuning against it counts as a violation.

Grading (100 points)

Component	Where graded	Points
Report	report.pdf , Sections 1, 3, 5, and References	20
Data handling	report.pdf , Section 2	20
Feature importance	report.pdf , Section 4	20
Method correctness	linear_gd.py	20
Test performance	predictions.csv	20

Unless noted otherwise, each criterion receives full credit when it is present, complete, and correct; partial credit when it is present but incomplete, unjustified, or partly incorrect; and no credit when it is missing.

Report (20 points)

Criterion	Points
1. Introduction states the task, the final validation RMSE, and the top three features	2
3.1 Cost function, gradient update, and stopping rule written correctly as equations	4
3.2 Validation split ratio and seed stated	2
3.3 Cost curves for at least two learning rates, with the chosen rate and convergence discussed	4

3.4 and 3.5 Experiment table with validation RMSE, and final model settings	4
5. Conclusion with a takeaway for AISD and one limitation	1
Format: required headings, labeled figures and tables, 6-page limit, references and AI-use statement	3

Data handling (20 points)

Criterion	Points
2.1 Summary statistics and missing-value counts for every column	3
2.2 Missing-value method, number affected, and justification	5
2.3 Data-entry errors: detection rule, count per column, handling, and justification	6
2.4 Scaling method, justification, and training-only statistics	4
2.5 Preprocessing order and consistent application to the test features	2

Feature importance (20 points)

Criterion	Points
4.1 At least two methods described	3
4.2 Score table covering every feature under every method, with a ranking based on the scores	4
4.3 Agreement and disagreement between methods explained	3
4.4 Rankings from at least three splits, with a stability conclusion	3
4.5 Top features plotted against the target; correlation matrix; effect of correlation on each method explained	4
4.6 Prediction vs. causation paragraph	3

Method correctness (20 points)

Criterion	Points
<code>LinearRegressionGD</code> constructor takes a learning rate, maximum iterations, and tolerance, and provides <code>fit</code> and <code>predict</code>	3
Gradient of the class cost function is correct for all parameters; on the same features, the fitted parameters match the closed-form least-squares solution to within a small tolerance	6
Stops when the change in cost falls below the tolerance or at the maximum iterations, and records the cost at every iteration	3
Preprocessing is leak-free: scaling and imputation statistics come from training data and are reused for the test features	3

Script runs end to end and reproduces <code>predictions.csv</code> from <code>LinearRegressionGD</code> (random seeds set)	3
Code is readable and commented, and adapted sources are cited	2

If you can't explain your code in a check-in, this may reduce your score.

Test performance (20 points)

Scored on the RMSE of `predictions.csv` against the true test labels. The target RMSE is 2.05. Each additional 0.15 RMSE, or any part of it, costs 1 point.

Points = $20 - \lceil (\text{RMSE} - 2.05) / 0.15 \rceil$, with a minimum of 0 and a maximum of 20.

Test RMSE	Points
2.05 or lower	20
Above 2.05, up to 2.20	19
Above 2.20, up to 2.35	18
Above 2.35, up to 2.50	17
Above 2.50, up to 2.65	16
Above 2.65, up to 2.80	15
Above 2.80, up to 2.95	14
Above 2.95, up to 3.10	13
Above 3.10, up to 3.25	12
Above 3.25, up to 3.40	11
Above 3.40, up to 3.55	10
Above 3.55, up to 3.70	9
Above 3.70, up to 3.85	8
Above 3.85, up to 4.00	7
Above 4.00, up to 4.15	6
Above 4.15, up to 4.30	5
Above 4.30, up to 4.45	4
Above 4.45, up to 4.60	3
Above 4.60, up to 4.75	2
Above 4.75, up to 4.90	1
Above 4.90	0

A `predictions.csv` that does not meet the format requirements in Part 2, or that was not produced by your `LinearRegressionGD` model, receives 0 points for this component.